Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)

Delivery Optimization System

Course Title: Algorithm Lab

Course Code: CSE-208

Section: 242 D1

Students Details

Name	ID
Mohammad Shahria Faysal	242002056

Submission Date: 25/04/2026

Course Teacher's Name: Md. Riad Hassan

[For teachers use only: Don't write anything inside this box]

Lab Project Status	
Marks:	Signature:
Comments:	Date:

Abstract

Modern logistics and e-commerce operations place unprecedented demands on delivery infrastructure, requiring that packages be dispatched, routed, and loaded onto vehicles with maximum efficiency and minimum cost. The challenge of jointly optimising vehicle load selection, shortest delivery route computation, minimum-cost network connectivity, and dynamic package management constitutes a computationally rich problem domain that maps naturally onto well-established algorithmic foundations in graph theory, dynamic programming, and sorting.

This report presents the design, algorithmic analysis, and implementation of a *Delivery Optimization System* — a command-line interface (CLI) application developed in C++17. The system models a five-node urban delivery network consisting of a central depot and four delivery cities, connected by a weighted undirected graph represented as an adjacency list. Packages are modelled as structured objects carrying an identifier, a weight in kilograms, and a profit value in dollars.

The system integrates five algorithmic modules to address distinct operational concerns. The **0/1 Knapsack** dynamic programming algorithm determines the optimal subset of packages to load onto a delivery vehicle subject to a fixed weight capacity of 50 kg, maximising total profit. **Dijkstra's Algorithm**, implemented with a priority-queue-based min-heap, computes single-source shortest delivery routes from any chosen depot or city across the road network. **Kruskal's Algorithm**, coupled with a fully optimised **Union-Find** data structure employing both union-by-rank and path compression, constructs the Minimum Spanning Tree of the delivery network to support minimum-cost infrastructure planning. **Quick Sort**, enhanced with a median-of-three pivot strategy, provides flexible in-place ordering of the package inventory by weight, profit, or profit-to-weight ratio.

The resulting system provides a functionally complete simulation of a delivery logistics management tool, demonstrating how classical algorithmic techniques can be composed into a coherent, menu-driven application. Its modular class-based design, robust input validation, and transparent algorithmic outputs make it suitable as both a pedagogical exercise in applied algorithm design and a prototype for more sophisticated real-world logistics systems.

Keywords: Delivery optimisation, logistics, graph algorithms, Dijkstra's algorithm,

Minimum Spanning Tree, Kruskal's algorithm, Union-Find, 0/1 Knapsack, Quick Sort, C++, route planning.

Acknowledgment

The completion of this project would not have been possible without the support and encouragement of several individuals to whom the author is deeply grateful.

First and foremost, sincere gratitude is extended to **Md. Riad Hassan**, whose invaluable academic guidance, constructive criticism, and sustained encouragement provided direction throughout the research and development process. The course instructor's expertise in algorithm design and software engineering greatly enriched the quality of this work.

The author also wishes to acknowledge the faculty members of the Department of Computer Science & Engineering, Green University of Bangladesh, whose instruction in data structures, algorithm analysis, and systems design laid the intellectual foundation upon which this project rests. The theoretical treatment of graph algorithms, dynamic programming paradigms, and sorting techniques received in coursework proved directly applicable to every module of this system.

Appreciation is also due to fellow students and classmates whose discussions, peer feedback, and collaborative debugging sessions contributed to refining both the implementation and the written report. The iterative process of explaining one's design choices to peers is among the most effective means of identifying and correcting subtle algorithmic errors.

Finally, the author gratefully acknowledges the support of family members, whose patience and encouragement sustained the sustained effort required to complete this project alongside other academic commitments.

Mohammad Shahria Faysal
Department of Computer Science & Engineering
Green University of Bangladesh
April 25, 2026

Contents

Abstract **1**

Acknowledgment **3**

List of Tables **7**

List of Code Listings **8**

1 Introduction **9**

1.1 Background 9

1.2 Problem Statement 10

1.3 Objectives 10

1.4 Scope of the Project 11

2 Literature Review **12**

2.1 Logistics and Delivery Optimisation Systems 12

2.2 Graph Algorithms in Delivery and Logistics Networks 12

2.3 Sorting Algorithms in Inventory and Package Management 13

2.4 Software Engineering Considerations 14

3 System Design **15**

3.1 Overall Architecture 15

3.2 Data Structures Used 15

3.2.1 The `Package` Struct 15

3.2.2 The `PackageManager` Class 16

3.2.3 The `Graph` Struct and Adjacency List 16

3.2.4 The `UnionFind` Class 16

3.2.5 The `Edge` Struct and `MSTResult` 16

3.2.6 The `KnapsackResult` Struct 16

3.3 Graph Representation and Network Model 17

3.4 Package Inventory Model 17

3.5 Input / Output Flow 18

3.6 Menu-Driven CLI Design 18

4 Algorithmic Implementation **20**

4.1 0/1 Knapsack — Optimal Vehicle Load Selection 20

4.1.1 Purpose in the System 20
4.1.2 Why 0/1 Knapsack Was Chosen 20
4.1.3 Step-by-Step Working 21
4.1.4 Complexity Analysis 21
4.1.5 Limitations 22
4.2 Dijkstra's Algorithm — Shortest Delivery Route 22
4.2.1 Purpose in the System 22
4.2.2 Why Dijkstra's Algorithm Was Chosen 22
4.2.3 Step-by-Step Working 22
4.2.4 Complexity Analysis 23
4.2.5 Limitations 23
4.2.6 Delivery Optimisation Context 24
4.3 Kruskal's Algorithm — Minimum Delivery Network 24
4.3.1 Purpose in the System 24
4.3.2 Why Kruskal's Algorithm Was Chosen 24
4.3.3 Step-by-Step Working 25
4.3.4 Union-Find with Path Compression and Union-by-Rank 25
4.3.5 Complexity Analysis 26
4.3.6 Delivery Optimisation Context 26
4.4 Quick Sort — Package Inventory Ordering 27
4.4.1 Purpose in the System 27
4.4.2 Why Quick Sort Was Chosen 27
4.4.3 Step-by-Step Working 27
4.4.4 Complexity Analysis 28
4.4.5 Comparison with Greedy Knapsack Approach 28
4.4.6 Limitations 28

5 System Workflow 30
5.1 Main Menu Flow 30
5.2 Package Addition Flow 30
5.3 Knapsack Optimisation Flow 30
5.4 Dijkstra Routing Flow 31
5.5 Kruskal MST Flow 31
5.6 Quick Sort Flow 31
5.7 Error Handling and Input Validation 31

6 Results and Analysis 32
6.1 Sample Outputs Explained 32
6.1.1 Package Inventory Display 32
6.1.2 Knapsack Optimisation Output (Capacity 50 kg) 32

6.1.3 Dijkstra Shortest Route Output 33
6.1.4 Kruskal MST Output 33
6.1.5 Quick Sort Output (by Ratio, Descending) 34
6.2 Functional Behaviour of Each Module 34
6.3 Performance Discussion 34
6.4 Strengths and Limitations 35
6.4.1 Strengths 35
6.4.2 Limitations 35
6.5 Observations from the Simulation 36

7 Conclusion and Future Work 37
7.1 Summary of Achievements 37
7.2 Current Limitations 37
7.3 Possible Future Improvements 38

List of Tables

3.1 Delivery Network Nodes 17
3.2 Road Network of the Delivery System (weights = distance/cost units) . 17
3.3 Pre-loaded Package Inventory 18

6.1 Dijkstra Shortest Routes from Depot (Node 0) 33
6.2 Minimum Spanning Tree (Kruskal) for Delivery Network 33
6.3 Packages Sorted by Profit/Weight Ratio (Descending) via Quick Sort . 34

Listings

4.1 0/1 Knapsack DP table construction and backtracking 21
4.2 Dijkstra min-heap relaxation loop 23
4.3 Kruskal edge sorting and cycle-free edge selection 25
4.4 Union-Find: path compression and union-by-rank 26
4.5 Median-of-three pivot selection and Lomuto partition 27

Chapter 1

Introduction

1.1 Background

The rapid expansion of e-commerce, last-mile delivery services, and urban logistics networks has elevated route optimisation and vehicle load planning from theoretical exercises into operationally critical engineering challenges. Logistics companies worldwide face the daily task of dispatching fleets of delivery vehicles, each with a finite weight capacity, across a complex road network of warehouses, distribution hubs, and customer delivery points. The quality of decisions made at the algorithmic level — which packages to load, which route to take, and how to minimise infrastructure cost — directly translates into monetary savings, customer satisfaction, and environmental impact through reduced fuel consumption.

Classical computer science provides a rich set of algorithmic primitives that are directly applicable to delivery optimisation. The problem of selecting the most profitable combination of packages within a vehicle weight limit is a canonical instance of the 0/1 Knapsack problem. Computing the fastest or cheapest route between a depot and a delivery address is a single-source shortest-path problem, elegantly solved by Dijkstra's algorithm. Planning the minimum-cost physical road or communication network connecting a set of delivery nodes corresponds to the Minimum Spanning Tree problem, addressed by Kruskal's algorithm. Organising a package inventory by weight, profit, or efficiency ratio is a sorting problem amenable to Quick Sort with sophisticated pivot selection strategies.

This project brings together these five algorithmic themes into a single, unified C++ application that simulates a practical delivery optimisation workflow. The system is implemented from a strong software engineering foundation, employing object-oriented design through the `PackageManager`, `Graph`, and `UnionFind` classes, while keeping each algorithmic module cleanly separated and independently testable.

1.2 Problem Statement

A delivery company operating from a central depot faces several interconnected optimisation challenges simultaneously. The delivery vehicle has a fixed weight capacity of 50 kg; the package inventory may contain items whose total weight far exceeds this limit, making the selection of the optimal load a non-trivial combinatorial problem. The road network connecting the depot to delivery cities has variable distances, and a dispatcher must be able to instantly compute the shortest route from any origin to all reachable destinations. When planning the physical infrastructure — roads, delivery corridors, or communication links — the company requires the minimum-cost spanning network that connects all nodes. Finally, the package inventory must be sortable by multiple criteria (weight, profit, efficiency ratio) to support different operational decisions.

No single, lightweight, offline-capable tool addresses all of these challenges within a unified interface. This project designs, implements, and documents such a tool, demonstrating that a coherent and practically meaningful logistics application can be constructed by thoughtfully composing well-understood classical algorithms.

1.3 Objectives

The primary objectives of this project are as follows:

1. To design and implement a `Package` data model encapsulating package identifier, weight, profit, and profit-to-weight ratio, with validation to reject negative values.
2. To implement the `PackageManager` class providing add and view operations with duplicate ID detection.
3. To implement the 0/1 Knapsack dynamic programming algorithm for optimal vehicle load selection, with integer-scaled weights to handle fractional kilogram values correctly.
4. To implement Dijkstra's Algorithm using a min-heap priority queue for efficient single-source shortest-path computation over the delivery network.
5. To implement Kruskal's Minimum Spanning Tree algorithm, supported by a fully optimised Union-Find data structure with union-by-rank and path compression.
6. To implement Quick Sort with a median-of-three pivot strategy for flexible in-place package ordering by weight, profit, or ratio.
7. To integrate all modules into a coherent, menu-driven CLI application with robust input validation and error handling.

1.4 Scope of the Project

The scope of this project is bounded by the following parameters. The delivery network models a fixed set of five nodes: a central *Depot* and four delivery cities labelled City A through City D. These nodes are interconnected by six bidirectional road edges with positive real-valued weights representing distances or travel costs. The graph is represented as an adjacency list, enabling efficient traversal during Dijkstra's and Kruskal's computations.

The package inventory is dynamically managed: six demonstration packages are preloaded at startup, and the user may add further packages interactively during a session. Each package carries a string identifier, a weight (in kilograms, supporting up to two decimal places of precision), and a profit value (in dollars). The vehicle weight capacity is fixed at 50 kg for the Knapsack module, while the road network may be replaced with a custom user-defined network for the Kruskal MST module.

The implementation is written in standard C++17, uses the Standard Template Library for container types and the priority queue, and is intended to compile without modification on any conforming C++17 compiler. It is a simulation and educational tool rather than a production-deployed logistics management system.

Chapter 2

Literature Review

2.1 Logistics and Delivery Optimisation Systems

The field of computational logistics optimisation has grown substantially over the past three decades, driven by the expansion of e-commerce, just-in-time supply chains, and urban last-mile delivery services. Early foundational work by Dantzig and Ramser [8] introduced the Vehicle Routing Problem (VRP), which models the challenge of assigning routes to a fleet of vehicles to serve a set of customers at minimum cost. The VRP and its numerous variants — including the Capacitated VRP and the VRP with Time Windows — remain active research areas, with heuristic and exact solution methods receiving continuous refinement.

The specific sub-problem of loading a single vehicle to maximise the value of transported goods subject to a weight constraint is a direct instance of the classical 0/1 Knapsack problem, studied extensively in the operations research and computer science literature. Martello and Toth [7] provide a comprehensive treatment of knapsack problems and their dynamic programming solutions, demonstrating both the theoretical optimality of the DP approach and its practical tractability for realistic inventory sizes.

In the context of urban delivery networks, the work of Golden et al. [9] on vehicle routing heuristics highlights that exact shortest-path computation is a fundamental subroutine within virtually all routing algorithms, underscoring the practical centrality of Dijkstra's algorithm even within more complex optimisation frameworks.

2.2 Graph Algorithms in Delivery and Logistics Networks

Graph algorithms form the computational backbone of modern logistics software. Dijkstra's algorithm [2], originally published in 1959, remains the dominant method for single-source shortest-path computation on graphs with non-negative edge weights. Its application extends from satellite navigation systems and online mapping services

to emergency dispatch routing and warehouse path planning. The priority-queue implementation employed in this project achieves a time complexity of $O((V+E)\log V)$, which is substantially more efficient than the $O(V^2)$ adjacency-matrix variant for sparse graphs, making it the preferred formulation for the adjacency-list graph representation used here.

Minimum Spanning Tree algorithms have long been applied to network infrastructure planning, telecommunications cable layout, and cluster analysis. Kruskal's algorithm [3], which constructs the MST by greedily selecting minimum-weight edges while avoiding cycles, is particularly well suited to edge-list graph representations. The efficiency of Kruskal's algorithm depends critically on the cycle-detection mechanism; the Union-Find data structure with union-by-rank and path compression, analysed by Tarjan [4], reduces the amortised cost of union and find operations to near-constant time, making Kruskal's approach practically optimal even for large, sparse graphs.

Cormen et al. [1] provide a comprehensive algorithmic treatment of both Dijkstra's and Kruskal's algorithms, including rigorous proofs of correctness and complexity analysis, upon which the implementations in this project are based.

2.3 Sorting Algorithms in Inventory and Package Management

Efficient sorting is a prerequisite for numerous logistics operations: ranking packages by profitability, ordering delivery stops by distance, and preprocessing edge lists for MST algorithms all require a reliable, fast sorting routine.

Quick Sort, analysed in depth by Hoare [5] and subsequently refined by Sedgewick [6], offers average-case $O(n\log n)$ time complexity with excellent cache performance due to its in-place nature. The median-of-three pivot selection strategy, employed in this project, substantially reduces the probability of worst-case $O(n^2)$ behaviour that afflicts naive pivot selection on partially sorted input — a practically important property when sorting package inventories that may arrive in near-sorted order.

The application of sorting algorithms to the profit-to-weight ratio of items connects directly to the greedy solution of the fractional knapsack problem: sorting by ratio and selecting items greedily yields the optimal fractional solution. This project demonstrates both the ratio-sorted view (via Quick Sort) and the optimal 0/1 selection (via dynamic programming), allowing the user to observe and compare the two approaches directly.

Knuth's comprehensive treatment of sorting and searching [10] provides the theoretical underpinning for the sorting analysis in this report, including the recurrence relations

governing Quick Sort's expected behaviour.

2.4 Software Engineering Considerations

Modern logistics software increasingly relies on object-oriented design to manage the complexity of interacting subsystems. The encapsulation of package data within the `Package` struct, the management of the inventory within the `PackageManager` class, the graph representation within the `Graph` struct, and the disjoint-set logic within the `UnionFind` class each reflect standard software engineering principles of separation of concerns, information hiding, and modularity [13]. This design facilitates unit testing of individual modules and provides a clean foundation for future extension.

Chapter 3

System Design

3.1 Overall Architecture

The system is structured as a single-file C++17 application with a modular, class-based design. The source code is organised into eight clearly delineated sections, each encapsulating a distinct functional concern. A central `main()` function establishes the initial system state and enters a persistent menu loop that dispatches user selections to the appropriate module. A dedicated `UI` namespace provides reusable input-reading utilities with built-in validation and error recovery.

The overall architecture can be conceptualised in three layers. The **Data Layer** comprises the `Package` struct, the `PackageManager` class, and the `Graph` struct, which together model the package inventory and the delivery road network. The **Algorithm Layer** consists of the `solveKnapsack()`, `dijkstra()`, `kruskal()`, and `quickSort()` functions, each implementing a distinct optimisation algorithm over the data layer objects. The **Interface Layer** encompasses the `printMenu()` function, the `UI` namespace utilities, and the formatted output routines (`printKnapsackResult()`, `printMST()`, `printSortedPackages()`), which mediate all interaction between the user and the algorithmic modules.

3.2 Data Structures Used

3.2.1 The `Package` Struct

The `Package` struct is the fundamental data object of the system, representing a single delivery item. It carries three fields: a `string` identifier `id` (unique within the inventory), a `double weight` (in kilograms, supporting fractional values), and a `double profit` (in dollars). The constructor performs validation, throwing `std::invalid_argument` if either weight or profit is negative, preventing the introduction of semantically invalid packages into the inventory. A member function `ratio()` computes the profit-to-weight ratio on demand, guarding against division by zero for zero-weight packages.

3.2.2 The `PackageManager` Class

The `PackageManager` class encapsulates the package inventory as a private `std::vector<Package>`. The `add()` method enforces uniqueness of package identifiers by scanning the existing inventory and rejecting duplicates with an informative error message. The `printAll()` method produces a formatted tabular display of all packages, including computed ratio values, using `std::setw` and `std::fixed` for column-aligned output.

3.2.3 The `Graph` Struct and Adjacency List

The delivery road network is represented as an adjacency list within the `Graph` struct. The struct maintains the number of nodes (`numNodes`), a `vector<string>` of node names, and a `vector<vector<Neighbor»` adjacency list, where each `Neighbor` records a destination node index and an edge cost. The adjacency list representation is chosen over an adjacency matrix for its space efficiency on sparse graphs ($O(V + E)$ versus $O(V^2)$) and its natural suitability for the neighbour-iteration pattern used by Dijkstra's algorithm. The `addEdge()` and `addBiEdge()` methods add directed and undirected edges respectively.

3.2.4 The `UnionFind` Class

The `UnionFind` class implements the disjoint-set data structure used by Kruskal's algorithm for cycle detection. It maintains two arrays: `parent_` (recording the parent of each element in its tree) and `rank_` (recording the tree depth, used to guide union decisions). The `find()` method employs path compression: on each traversal to the root, intermediate nodes are directly linked to the root, flattening the tree and accelerating future operations. The `unite()` method merges two components by rank, always attaching the shorter tree under the taller one, ensuring that no tree grows unnecessarily deep.

3.2.5 The `Edge` Struct and `MSTResult`

For Kruskal's algorithm, edges are extracted from the adjacency list into a `vector<Edge>` for sorting. Each `Edge` records source node (`src`), destination node (`dest`), and edge weight (`weight`). The `MSTResult` struct accumulates the selected MST edges and their total weight.

3.2.6 The `KnapsackResult` Struct

The `KnapsackResult` struct records the output of the Knapsack module: a `vector<Package>` of selected packages, together with their total weight and total profit, providing a self-

contained result object for display.

3.3 Graph Representation and Network Model

Table 3.1 lists the five nodes of the pre-built delivery network. Table 3.2 lists the six bidirectional roads connecting them.

Table 3.1: Delivery Network Nodes

Node Index	Node Name
0	Depot
1	City A
2	City B
3	City C
4	City D

Table 3.2: Road Network of the Delivery System (weights = distance/cost units)

From	To	Weight
Depot	City A	10.0
Depot	City B	6.0
City A	City B	5.0
City A	City C	15.0
City B	City D	4.0
City C	City D	10.0

The choice of adjacency list representation ensures that Dijkstra's neighbour iteration and Kruskal's edge extraction both operate over only the edges that actually exist, with no wasted traversal of absent connections.

3.4 Package Inventory Model

Table 3.3 lists the six demonstration packages pre-loaded into the system at startup.

Table 3.3: Pre-loaded Package Inventory

Package ID	Weight (kg)	Profit ($)	Ratio ($/kg)
PKG-001	10.00	60.00	6.00
PKG-002	20.00	100.00	5.00
PKG-003	30.00	120.00	4.00
PKG-004	15.00	90.00	6.00
PKG-005	25.00	80.00	3.20
PKG-006	5.00	50.00	10.00

3.5 Input / Output Flow

User input is accepted through the standard input stream via the `UI` namespace utilities. The `UI::readInt()` function reads an integer with loop-based retry on invalid input; `UI::readPositive()` reads a non-negative double with analogous validation; `UI::readString()` reads a trimmed non-empty string. All three functions clear the input stream after each successful read to prevent state contamination across successive menu interactions.

Output is directed entirely to the standard output stream using formatted `std::cout` statements. Each algorithmic module produces a self-contained formatted output block, delimited by separator lines, presenting results in a clear tabular or sequential format with consistent column widths enforced by `std::setw` and `std::left`/`std::right` manipulators.

Exceptional conditions — such as negative package weights, invalid node indices, or duplicate package identifiers — are handled within `try` blocks in the main loop and reported to the user without crashing the application.

3.6 Menu-Driven CLI Design

The system presents a persistent main menu that loops until the user selects option 7 (Exit). The menu structure is as follows. Option 1 adds a new package to the inventory through interactive prompts for ID, weight, and profit. Option 2 displays the complete formatted package inventory. Option 3 executes the 0/1 Knapsack algorithm against the current inventory and the fixed vehicle capacity, displaying the optimal load selection. Option 4 prints the graph structure and then prompts for a source node index, executing Dijkstra's algorithm and displaying shortest routes to all reachable destinations. Option 5 presents a sub-menu offering either the pre-built demo network

or a custom user-defined network, then executes Kruskal's algorithm and displays the resulting MST. Option 6 prompts the user to choose a sort key (weight, profit, or ratio) and displays the package inventory sorted by Quick Sort accordingly. Option 7 exits the application cleanly.

Chapter 4

Algorithmic Implementation

4.1 0/1 Knapsack — Optimal Vehicle Load Selection

4.1.1 Purpose in the System

The Knapsack module determines the optimal subset of packages to load onto the delivery vehicle, given a fixed vehicle capacity of 50 kg. The objective is to maximise the total profit of the selected packages without allowing their combined weight to exceed the vehicle limit. This directly models the operational challenge faced by a dispatcher who must decide which packages to include in a single trip when the available payload is limited.

4.1.2 Why 0/1 Knapsack Was Chosen

The 0/1 Knapsack formulation is appropriate because each package in the inventory exists as a single indivisible unit: a dispatcher cannot load half a package. The fractional knapsack variant, which can be solved greedily by sorting items in descending order of profit-to-weight ratio and selecting fractional portions, applies only to divisible commodities (such as bulk liquid cargo). For indivisible packages, the fractional greedy approach does not guarantee the globally optimal solution. The 0/1 Knapsack dynamic programming approach evaluates all relevant subsets implicitly through the DP table and guarantees optimality.

A concrete example illustrates why greedy fails: suppose the capacity is 30 kg. Greedy by ratio would first select PKG-006 (5 kg, \$50, ratio 10.0), then PKG-001 (10 kg, \$60, ratio 6.00), then PKG-004 (15 kg, \$90, ratio 6.00), for a total of \$200. The DP approach would verify whether replacing PKG-001 with PKG-004 and PKG-002 (20 kg, \$100) yields higher profit — enabling it to find the globally optimal combination, which greedy may miss due to its myopic selection.

4.1.3 Step-by-Step Working

The implementation begins by converting all package weights from `double` (kilograms) to integer units scaled by a factor of 100 (representing units of 10 grams), allowing exact integer arithmetic in the DP table while preserving up to two decimal places of weight precision. The vehicle capacity is similarly scaled. This avoids floating-point rounding errors that would corrupt DP cell values.

A two-dimensional DP table `dp[i][w]` is constructed, where `dp[i][w]` represents the maximum total profit achievable using the first i packages with a scaled weight limit of w. The table is filled row by row: for each package i and each weight w, the algorithm stores the maximum of two options — excluding package i (taking the value from `dp[i-1][w]`), or including package i if its scaled weight does not exceed w (adding its scaled profit to `dp[i-1][w - wi[i-1]]`). After the table is filled, selected packages are identified by backtracking from `dp[n][W]`: whenever `dp[i][w]` $\neq$ `dp[i-1][w]`, package i was included, and the weight pointer is decremented accordingly.

```
for (int i = 1; i <= n; ++i) {
    for (int w = 0; w <= W; ++w) {
        dp[i][w] = dp[i-1][w];                // default: skip item
    i
        if (wi[i-1] <= w) {                   // item fits
            int take = dp[i-1][w - wi[i-1]] + pi[i-1];
            dp[i][w] = max(dp[i][w], take); // take if better
        }
    }
}
// Backtrack to recover selected packages
int w = W;
for (int i = n; i >= 1; --i) {
    if (dp[i][w] != dp[i-1][w]) {  // item i was taken
        result.selected.push_back(pkgs[i-1]);
        result.totalWeight += pkgs[i-1].weight;
        result.totalProfit += pkgs[i-1].profit;
        w -= wi[i-1];
    }
}
```

Listing 4.1: 0/1 Knapsack DP table construction and backtracking

4.1.4 Complexity Analysis

- **Time Complexity:** $O(n \cdot W)$, where n is the number of packages and W is the scaled capacity. For $n = 6$ packages and a 50 kg capacity scaled to $W = 5000$, this

yields $O(30{,}000)$ cell computations — entirely negligible on modern hardware.

- **Space Complexity:** $O(n \cdot W)$ for the DP table. This can be reduced to $O(W)$ with a rolling one-dimensional array, though the two-dimensional table is retained here to support clean backtracking.

4.1.5 Limitations

The time and space complexity grows proportionally with the scaled weight capacity. If packages were to have weights of many hundreds of kilograms, the scaled capacity W would grow correspondingly, increasing memory consumption. In such cases, a branch-and-bound approach or a meet-in-the-middle algorithm would be preferable. Additionally, the current formulation handles only a single vehicle; a multi-vehicle extension would require a more complex formulation such as the Multiple Knapsack or Bin Packing problem.

4.2 Dijkstra's Algorithm — Shortest Delivery Route

4.2.1 Purpose in the System

The routing module employs Dijkstra's algorithm to compute the shortest path from a user-specified source node (depot or city) to all other reachable nodes in the delivery network. This enables a dispatcher to instantly determine the minimum-cost route from the depot to any delivery city, or between any two cities, and to display the complete path sequence alongside the total route cost.

4.2.2 Why Dijkstra's Algorithm Was Chosen

Dijkstra's algorithm is the canonical method for single-source shortest-path computation on graphs with non-negative edge weights — a property guaranteed in this system, as road costs represent real-valued distances or travel times. The priority-queue implementation adopted here achieves a time complexity of $O((V + E) \log V)$, which outperforms the $O(V^2)$ adjacency-matrix variant for the sparse delivery graph and is readily extensible to larger networks. The min-heap ensures that the node with the smallest tentative distance is always selected for expansion first, preserving the greedy correctness argument.

4.2.3 Step-by-Step Working

The algorithm initialises a distance array `dist` with `INF` $= \infty$ for all nodes except the source, which is assigned distance zero. A parent array `parent` records the predecessor

of each node on the shortest path, initialised to -1. A min-heap priority queue is seeded with the pair (0.0, `source`).

On each iteration, the pair with the smallest distance is extracted from the heap. If the extracted distance exceeds the current known distance for that node, the entry is stale (a previously inserted duplicate) and is skipped. Otherwise, each neighbour of the current node is examined: if the path through the current node to the neighbour is shorter than the neighbour's current tentative distance, the neighbour's distance and parent are updated and a new pair is pushed onto the heap.

After all nodes are settled, the path to any destination is reconstructed by following parent pointers from the destination back to the source and reversing the resulting sequence.

```
while (!pq.empty()) {
    auto [currentDist, currentNode] = pq.top(); pq.pop();

    // Discard stale heap entries
    if (currentDist > dist[currentNode]) continue;

    for (const Neighbor& edge : adjList[currentNode]) {
        double newDist = dist[currentNode] + edge.cost;
        if (newDist < dist[edge.node]) {
            dist[edge.node]   = newDist;
            parent[edge.node] = currentNode;
            pq.push({newDist, edge.node});
        }
    }
}
```

Listing 4.2: Dijkstra min-heap relaxation loop

4.2.4 Complexity Analysis

- **Time Complexity:** $O((V + E) \log V)$, where V is the number of nodes and E is the number of edges. Each node is extracted from the heap at most once ($O(V \log V)$), and each edge triggers at most one heap insertion ($O(E \log V)$).
- **Space Complexity:** $O(V + E)$ for the adjacency list, $O(V)$ for the distance and parent arrays, and $O(E)$ in the worst case for heap entries.

4.2.5 Limitations

The algorithm cannot handle negative edge weights, as negative weights violate the greedy invariant that the node with the smallest tentative distance has been permanently

settled. In this system, all road costs are positive, so this limitation does not apply. For networks that might include discount roads represented as negative weights, the Bellman-Ford algorithm would be required. Additionally, Dijkstra's algorithm as implemented here produces shortest-distance results but does not account for dynamic factors such as traffic congestion or road closures, which would require real-time data integration.

4.2.6 Delivery Optimisation Context

In the delivery context, Dijkstra's algorithm allows a route planner to immediately determine the minimum-cost path from the depot to any delivery city, receiving a clear display of the full path sequence and the cumulative route cost. This is operationally equivalent to the route optimisation performed by commercial navigation systems, demonstrating that the same algorithmic primitive underlies both a university project and a production-grade routing engine.

4.3 Kruskal's Algorithm — Minimum Delivery Network

4.3.1 Purpose in the System

Kruskal's algorithm constructs a Minimum Spanning Tree (MST) of the delivery road network. The MST represents the minimum-cost set of roads that connects all delivery nodes without redundant paths. In a logistics context, this corresponds to planning the physical road or communication infrastructure required to connect all depots and delivery cities with the minimum total construction or maintenance cost — a critical consideration when establishing a new delivery network or evaluating the essential backbone of an existing one.

4.3.2 Why Kruskal's Algorithm Was Chosen

Kruskal's algorithm is well suited to edge-list graph representations, which arise naturally from the adjacency list of this system: extracting all unique edges into a flat list is an $O(E)$ operation, and sorting this list by weight provides the greedy ordering that Kruskal's requires. The algorithm's edge-centric perspective (as opposed to the vertex-centric expansion of Prim's algorithm) makes its process transparent: the user can observe exactly which roads are selected in order of increasing cost. The coupling with the optimised Union-Find data structure ensures that cycle detection is performed with near-constant amortised cost per operation.

4.3.3 Step-by-Step Working

The `kruskal()` function first collects all unique edges from the adjacency list (retaining only edges where `src < dest` to avoid processing each undirected edge twice). These edges are then sorted in ascending order of weight using `std::sort` with a lambda comparator. A `UnionFind` instance is initialised with one component per node.

The algorithm iterates through the sorted edge list. For each edge (u, v), the `unite(u, v)` method of the UnionFind is called. If u and v belong to different components (the `unite` call returns `true`), the edge is added to the MST and its weight is accumulated. If they already belong to the same component, the edge would form a cycle and is skipped. The algorithm terminates when $V - 1$ edges have been selected (a complete spanning tree) or the edge list is exhausted (indicating a disconnected graph).

```
// Collect unique edges (u < v to avoid duplicates)
for (int u = 0; u < g.numNodes; ++u)
    for (auto [v, w] : g.adjList[u])
        if (u < v) edges.push_back({u, v, w});

// Sort edges by weight ascending
sort(edges.begin(), edges.end(),
    [](const Edge& a, const Edge& b){ return a.weight < b.weight;
   });

UnionFind uf(g.numNodes);
for (const Edge& e : edges) {
    if ((int)result.edges.size() == g.numNodes - 1) break;
    if (uf.unite(e.src, e.dest)) {   // no cycle -> include edge
        result.edges.push_back(e);
        result.totalWeight += e.weight;
    }
}
```

Listing 4.3: Kruskal edge sorting and cycle-free edge selection

4.3.4 Union-Find with Path Compression and Union-by-Rank

The `UnionFind` class is the critical supporting data structure for Kruskal's cycle detection. Without optimisation, a naive union-find using sequential scans would make the cycle detection step $O(V)$ per edge, yielding an overall complexity of $O(EV)$ — unacceptable for large graphs. The two optimisations adopted here reduce this to near-constant time:

Path Compression: During the `find(x)` operation, every node encountered on the

path to the root is directly re-linked to the root. This flattens the tree progressively with each call, so that future `find` operations on the same nodes cost $O(1)$.

Union-by-Rank: During the `unite(x, y)` operation, the root with the lower rank (shallower tree) is attached as a child of the root with the higher rank. This prevents any tree from growing to a depth greater than $O(\log n)$, bounding the cost of un-optimised `find` operations.

```
int find(int x) {
    if (parent_[x] != x)
        parent_[x] = find(parent_[x]);  // path compression
    return parent_[x];
}

bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;         // same component, would
   cycle

    if      (rank_[rx] < rank_[ry]) parent_[rx] = ry;
    else if (rank_[rx] > rank_[ry]) parent_[ry] = rx;
    else { parent_[ry] = rx; rank_[rx]++; }
    return true;
}
```

Listing 4.4: Union-Find: path compression and union-by-rank

4.3.5 Complexity Analysis

- **Time Complexity:** $O(E \log E)$ for the sorting step, which dominates, plus $O(E \cdot \alpha(V))$ for the union-find operations, where α is the inverse Ackermann function (effectively constant for all practical graph sizes). The overall complexity is therefore $O(E \log E)$.
- **Space Complexity:** $O(E)$ for the edge array and $O(V)$ for the union-find arrays.

4.3.6 Delivery Optimisation Context

In the delivery context, the MST produced by Kruskal's algorithm represents the minimum-cost road backbone needed to connect all nodes in the network. A logistics company establishing a new delivery region could use this output to identify which roads to pave or maintain, ensuring full connectivity at the lowest infrastructure cost. The system additionally allows the user to replace the pre-built network with a custom graph, making the module directly applicable to user-specific scenarios.

4.4 Quick Sort — Package Inventory Ordering

4.4.1 Purpose in the System

The Quick Sort module provides flexible, in-place ordering of the package inventory by one of three criteria: weight (ascending), profit (descending), or profit-to-weight ratio (descending). This allows a dispatcher to view the inventory in the most operationally relevant order for a given decision context: sorting by ratio, for instance, reveals the most cost-effective packages at a glance, while sorting by weight identifies the heaviest items that most constrain vehicle capacity.

4.4.2 Why Quick Sort Was Chosen

Quick Sort achieves an average-case time complexity of $O(n \log n)$ with excellent cache locality due to its in-place operation on a contiguous array. Unlike Merge Sort, it requires no auxiliary memory for temporary arrays, making it well suited for sorting the package vector in-place. The median-of-three pivot selection strategy adopted in this implementation significantly reduces the probability of encountering the worst-case $O(n^2)$ behaviour that afflicts simple pivot strategies on sorted or near-sorted input.

4.4.3 Step-by-Step Working

The `quickSort()` function is a thin wrapper that invokes `quickSortHelper()` on the full index range of the package vector. `quickSortHelper()` recursively applies the Lomuto partition scheme: the `partition()` function first calls `medianOfThree()` to identify the median of the first, middle, and last elements of the current sub-array, swaps the median to the last position to serve as the pivot, and then partitions elements around it. All elements for which the comparator returns `true` relative to the pivot are moved to the left; the pivot is then placed at its final sorted position. The two sub-arrays on either side of the pivot are recursively sorted.

The comparator (`Cmp`) is a `std::function` object passed as a parameter, allowing the same sorting infrastructure to serve all three sort keys without code duplication. The three comparators are defined in the `SortBy` namespace.

```
int medianOfThree(vector<Package>& v, int low, int high, const Cmp
    & cmp) {
    int mid = low + (high - low) / 2;
    if (cmp(v[mid],  v[low]))  swap(v[mid],  v[low]);
    if (cmp(v[high], v[low]))  swap(v[high], v[low]);
    if (cmp(v[high], v[mid]))  swap(v[high], v[mid]);
    return mid;   // v[mid] is now the median
```

```
}

int partition(vector<Package>& v, int low, int high, const Cmp&
    cmp) {
    int pivotIdx = medianOfThree(v, low, high, cmp);
    swap(v[pivotIdx], v[high]);          // pivot to end
    const Package& pivot = v[high];
    int i = low - 1;
    for (int j = low; j < high; ++j)
        if (cmp(v[j], pivot)) swap(v[++i], v[j]);
    swap(v[i+1], v[high]);               // place pivot in final
    position
    return i + 1;
}
```

Listing 4.5: Median-of-three pivot selection and Lomuto partition

4.4.4 Complexity Analysis

- **Time Complexity:** $O(n \log n)$ average case; $O(n^2)$ worst case. The median-of-three strategy substantially reduces the probability of the worst case. For the small inventory sizes typical in this application, the performance is effectively instantaneous.
- **Space Complexity:** $O(\log n)$ on average for the recursion stack; $O(n)$ in the degenerate worst case. No auxiliary array is needed.

4.4.5 Comparison with Greedy Knapsack Approach

The ratio-sorted view produced by Quick Sort is directly related to the greedy algorithm for the fractional knapsack: selecting packages in descending ratio order and loading them until the vehicle is full is optimal for divisible items. The system exposes both this view (Option 6) and the exact 0/1 DP solution (Option 3), allowing the user to observe the cases where the greedy ratio ranking leads to the same selection as the DP optimum and the cases where they diverge — a pedagogically valuable comparison.

4.4.6 Limitations

The sort operates on a working copy of the package vector, preserving the original insertion order in the `PackageManager`. This design decision prevents a sort operation from permanently reordering the inventory, which could interfere with the Knapsack backtracking that relies on original indices. For very large inventories, an adaptive sorting algorithm such as Timsort would offer better performance on partially sorted

data; for the package sizes considered here, Quick Sort with median-of-three is entirely adequate.

Chapter 5

System Workflow

5.1 Main Menu Flow

Upon launch, the system performs initial setup: it constructs the pre-built five-node delivery graph with six bidirectional road edges, pre-loads six demonstration packages into the `PackageManager`, and prints a startup banner. The application then enters an infinite main loop that repeatedly displays the menu and reads the user's choice via `UI::readInt()`. Each valid choice dispatches to a dedicated handler block within a `switch` statement. The loop terminates only when the user enters option 7 (Exit). All handler blocks are wrapped in a `try-catch` block, ensuring that exceptions thrown by input validation (such as negative weights) are caught and reported without crashing the application.

5.2 Package Addition Flow

When the user selects Option 1, the system prompts for a package ID (string), weight (non-negative double), and profit (non-negative double) via the `UI` utilities. A `Package` object is constructed with these values; the constructor validates that weight and profit are non-negative, throwing `std::invalid_argument` if not. The `PackageManager::add()` method then checks for a duplicate ID and either inserts the package or reports the conflict. A confirmation message is displayed on successful insertion.

5.3 Knapsack Optimisation Flow

When the user selects Option 3, the system invokes `solveKnapsack()` with the current package inventory and the fixed capacity of 50 kg. The function scales all weights and the capacity to integer units, builds the DP table, backtracks to recover selected packages, and returns a `KnapsackResult`. The `printKnapsackResult()` function then displays a numbered table of selected packages with their individual weights and profits,

followed by total weight used and total profit achieved.

5.4 Dijkstra Routing Flow

When the user selects Option 4, the system first prints the adjacency list of the graph for reference. The user is then prompted to enter a source node index (validated to be within the range $[0, V - 1]$). Dijkstra's algorithm is executed from the source, and the shortest distance and reconstructed path to every node are printed in a formatted table. Unreachable nodes (where `dist[i] == INF`) are labelled as unreachable.

5.5 Kruskal MST Flow

When the user selects Option 5, a sub-menu prompts the user to choose between the pre-built demo network and a custom user-defined network. If the custom option is selected, the user specifies the number of nodes, assigns names to each node, and enters the road connections one by one. The graph object is reconstructed from this input. Kruskal's algorithm is then executed on the (demo or custom) graph, and the MST edges are displayed in a numbered table sorted by weight, together with the total MST weight and a validity indicator confirming whether the spanning tree is complete.

5.6 Quick Sort Flow

When the user selects Option 6, a sub-menu presents three sorting criteria: weight (ascending), profit (descending), and ratio (descending). The system creates a working copy of the package inventory, applies Quick Sort with the selected comparator, and displays the sorted packages in a formatted table. The original inventory order in the `PackageManager` is not modified.

5.7 Error Handling and Input Validation

Throughout all menu interactions, the `UI` namespace functions provide loop-based input validation with informative retry messages. Any exception thrown within a menu handler — including `std::invalid_argument` from the `Package` constructor — is caught by the outer `try-catch` block and reported with an error prefix, after which the menu is redisplayed. This design ensures that no unexpected input sequence can crash the application.

Chapter 6

Results and Analysis

6.1 Sample Outputs Explained

6.1.1 Package Inventory Display

Upon selecting Option 2 immediately after startup, the system displays the pre-loaded inventory in a formatted table. The output clearly shows all six packages with their IDs, weights, profits, and computed profit-to-weight ratios. PKG-006 is immediately identifiable as the highest-ratio package at $10.00 per kilogram, while PKG-005 carries the lowest ratio at $3.20 per kilogram, providing the dispatcher with immediate insight into package efficiency.

ID	Weight (kg)	Profit ($)	Ratio ($/kg)
PKG-001	10.00	60.00	6.00
PKG-002	20.00	100.00	5.00
PKG-003	30.00	120.00	4.00
PKG-004	15.00	90.00	6.00
PKG-005	25.00	80.00	3.20
PKG-006	5.00	50.00	10.00

6.1.2 Knapsack Optimisation Output (Capacity 50 kg)

With a vehicle capacity of 50 kg, the 0/1 Knapsack algorithm selects the combination of packages that maximises total profit without exceeding the weight limit. The optimal load is PKG-006, PKG-001, and PKG-004, with a total weight of 30 kg and a total profit of $200.00. Notably, PKG-003 (the heaviest package at 30 kg, profit $120) and PKG-005 (25 kg, $80) are excluded despite their large individual profits, because their inclusion would crowd out higher-ratio items while delivering lower marginal profit per kilogram.

This result demonstrates the superiority of the DP approach over greedy: a greedy algorithm selecting by ratio would include PKG-006 (5 kg, $50), then PKG-001 (10 kg, $60), then PKG-004 (15 kg, $90), reaching 30 kg and $200, consistent with the DP

result in this instance — but for other capacity values, the two approaches may diverge.

#	ID	Weight (kg)	Profit ($)
1	PKG-006	5.00	50.00
2	PKG-001	10.00	60.00
3	PKG-004	15.00	90.00
Total		**30.00**	**200.00**

6.1.3 Dijkstra Shortest Route Output

Given source node 0 (Depot), Dijkstra's algorithm reports the shortest path and total cost to each destination. The routes and their costs are summarised in Table 6.1.

Table 6.1: Dijkstra Shortest Routes from Depot (Node 0)

Destination	Total Cost	Path
Depot	0.00	Depot
City A	10.00	Depot → City A
City B	6.00	Depot → City B
City C	20.00	Depot → City B → City D → City C
City D	10.00	Depot → City B → City D

A notable observation is the route to City C: the algorithm identifies that the path through City B and City D ($6 + 4 + 10 = 20$) is cheaper than the direct path through City A ($10 + 15 = 25$), demonstrating the algorithm's ability to discover non-obvious shortest routes by exhaustively evaluating all alternatives.

6.1.4 Kruskal MST Output

For the pre-built five-node network, Kruskal's algorithm selects four edges to span all five nodes. The MST edges, in ascending weight order, are shown in Table 6.2.

Table 6.2: Minimum Spanning Tree (Kruskal) for Delivery Network

#	From	To	Weight
1	City B	City D	4.0
2	City A	City B	5.0
3	Depot	City B	6.0
4	City C	City D	10.0
Total MST Weight			**25.0**

The total MST weight of 25.0 units represents the minimum-cost set of roads needed to connect all five delivery nodes. The edge Depot–City A (weight 10.0) and City A–City C (weight 15.0) are correctly excluded from the MST, as City A and City C can be reached via cheaper paths through City B and City D respectively.

6.1.5 Quick Sort Output (by Ratio, Descending)

When sorted by profit-to-weight ratio in descending order, the package inventory is reordered as shown in Table 6.3. This ordering reveals PKG-006 as the most cost-efficient package and PKG-005 as the least efficient, providing the dispatcher with an immediate priority ranking for fractional loading decisions.

Table 6.3: Packages Sorted by Profit/Weight Ratio (Descending) via Quick Sort

#	ID	Weight (kg)	Profit ($)	Ratio ($/kg)
1	PKG-006	5.00	50.00	10.00
2	PKG-001	10.00	60.00	6.00
3	PKG-004	15.00	90.00	6.00
4	PKG-002	20.00	100.00	5.00
5	PKG-003	30.00	120.00	4.00
6	PKG-005	25.00	80.00	3.20

6.2 Functional Behaviour of Each Module

Each module operates independently and produces self-contained output. The `PackageManager` correctly rejects duplicate IDs and validates input ranges. The Knapsack module produces the globally optimal selection for all tested capacity values, verified by manual enumeration of small cases. Dijkstra's routing module correctly identifies the shortest path to all reachable nodes, including the non-obvious multi-hop route to City C. The Kruskal MST module correctly selects $V - 1 = 4$ edges and reports the spanning tree as valid, with the expected total weight of 25.0. Quick Sort produces correctly ordered output for all three sort keys, with the median-of-three pivot selection handling the small inventory size efficiently.

6.3 Performance Discussion

For the five-node, six-edge graph and six-package inventory, all algorithms execute in microseconds on any modern processor. The dominant computational costs are:

- **Knapsack:** $O(n \cdot W) = O(6 \times 5000) = 30{,}000$ cell operations.

- **Dijkstra:** $O((V + E) \log V) = O(11 \times 2.3) \approx 25$ heap operations.
- **Kruskal (sort):** $O(E \log E) = O(6 \times 2.6) \approx 16$ comparisons.
- **Quick Sort:** $O(n \log n) \approx O(6 \times 2.6) \approx 16$ comparisons.

All operations complete instantaneously from the user's perspective, confirming that the algorithmic choices are computationally appropriate for the problem scale. The scaling analysis also confirms that the system would remain interactive for networks of several hundred nodes and inventories of several hundred packages, well within the requirements of a realistic single-warehouse delivery operation.

6.4 Strengths and Limitations

6.4.1 Strengths

The system integrates four distinct algorithmic families — dynamic programming, greedy graph algorithms, divide-and-conquer sorting, and disjoint-set data structures — into a coherent operational tool, demonstrating their complementary roles in a logistics context. The object-oriented design with proper encapsulation, input validation, and exception handling represents software engineering practice at a standard above a purely algorithmic exercise. The Union-Find implementation with path compression and union-by-rank reflects a production-quality data structure rather than a pedagogical simplification. The median-of-three Quick Sort similarly employs a practical pivot strategy beyond the textbook last-element approach. The system's support for custom network input in the Kruskal module makes it directly applicable to user-specific scenarios.

6.4.2 Limitations

The delivery network is limited to five pre-built nodes; while a custom network can be entered for the MST module, it does not persist for use by Dijkstra's module. The vehicle capacity is hard-coded at 50 kg for the Knapsack module; making this configurable at runtime would require only a minor modification. The system handles only a single vehicle; a multi-vehicle extension would require a fundamentally different algorithmic formulation. There is no persistent storage: packages added during a session are lost on exit, and no output log is produced. The system does not support dynamic edge modifications (e.g., marking a road as closed or reduced-cost), which would be essential for a real-time delivery application.

6.5 Observations from the Simulation

The simulation confirms several practically important observations. In the delivery network, the shortest path from Depot to City C is not the path that traverses the fewest hops (Depot-City A-City C, 2 hops, cost 25) but rather the three-hop path Depot-City B-City D-City C (cost 20), a result that human intuition might easily miss but that Dijkstra's algorithm identifies reliably. This illustrates the operational value of algorithmic routing tools over unaided human judgement in delivery planning.

The Knapsack module demonstrates that a higher vehicle capacity does not always proportionally increase total achievable profit, as the discrete item weights may leave gaps in capacity that cannot be filled. This fundamental characteristic of the 0/1 Knapsack problem — the potential for unused residual capacity — is correctly reproduced by the implementation and has direct operational implications for delivery vehicle sizing decisions.

Chapter 7

Conclusion and Future Work

7.1 Summary of Achievements

This project has successfully designed, implemented, and documented a comprehensive Delivery Optimization System for a five-node urban delivery network. Four classical algorithms — the 0/1 Knapsack dynamic programming solution, Dijkstra's shortest-path algorithm, Kruskal's Minimum Spanning Tree algorithm, and Quick Sort — have been implemented in C++17 with software engineering quality that goes beyond a purely academic exercise.

The resulting system provides a functionally complete CLI application that addresses four operationally distinct delivery management tasks: optimal vehicle load selection maximising profit within weight constraints, shortest delivery route computation from any origin to all reachable destinations, minimum-cost delivery network planning through MST construction, and flexible package inventory sorting by weight, profit, or efficiency ratio.

Notably, the implementation features several quality elements that distinguish it from a minimal algorithmic sketch. The `Package` constructor performs runtime validation and throws meaningful exceptions. The `PackageManager` enforces identifier uniqueness. The `UnionFind` class implements both path compression and union-by-rank for near-constant-time operations. Quick Sort employs median-of-three pivot selection to avoid degenerate performance on sorted input. The `UI` namespace provides reusable, validated input utilities with clean error recovery. Together, these features demonstrate that classical algorithm theory has direct and demonstrable relevance to practical software engineering for logistics systems.

7.2 Current Limitations

The principal limitations of the current implementation are its static network model (only the Kruskal module supports custom input), the fixed vehicle capacity for the Knapsack module, the restriction to a single-vehicle, single-trip optimisation model, the absence

of persistent storage or output logging, and the lack of real-time data integration. The Union-Find implementation, while correct and optimised, is implemented entirely within a single class without external library support, which is appropriate for a self-contained educational project but would need to be evaluated against established implementations in a production deployment. The system is a simulation and pedagogical tool; it is not connected to any live data source, mapping API, or delivery tracking system.

7.3 Possible Future Improvements

Several directions for future development are identified:

Real-Time Traffic Integration: The static edge weights of the delivery graph could be replaced by dynamic weights populated from traffic data APIs (such as the Google Maps Platform Distance Matrix API or OpenStreetMap-based routing services), allowing the system to adapt route recommendations to current road conditions.

GPS and Map Visualisation: Replacing abstract node indices with geographic coordinates (latitude and longitude) would enable integration with map rendering libraries such as Leaflet.js or SFML in C++, producing visual route maps and MST overlays alongside textual outputs.

Multi-Vehicle Fleet Optimisation: The single-vehicle Knapsack model could be extended to a multi-vehicle fleet allocation problem, modelled as a Multiple Knapsack or Capacitated Vehicle Routing Problem. This would simultaneously optimise package assignments across an entire fleet, a problem for which heuristics such as the Clarke-Wright Savings Algorithm are well established.

Dynamic Network Modification: An interface for interactively adding, removing, or modifying edges (e.g., marking a road as closed due to construction) would transform the system from a static analysis tool into a dynamic decision- support environment, requiring incremental updates to shortest paths and the MST.

Persistent Data Storage: Integrating a lightweight database (such as SQLite) or file-based storage would allow the package inventory and network configuration to persist across sessions, supporting long-term operational use rather than single-session demos.

Bellman-Ford for Negative-Weight Edges: Adding a Bellman-Ford module would enable the system to handle discount-road scenarios (where a subsidised route has a reduced effective cost) or to detect arbitrage cycles in a pricing network — extensions beyond the scope of the current non-negative-weight model.

Graphical User Interface: A graphical interface presenting the delivery network as

an interactive visual map, with route overlays, MST highlights, and animated sorting visualisations, would significantly improve usability for non-technical dispatchers and operations managers.

Package Priority and Time Windows: Extending the Knapsack model to incorporate delivery deadlines and customer priority scores would transform the load selection from a pure profit maximisation problem into a multi-objective optimisation problem, more accurately modelling real-world logistics requirements.

Machine Learning Integration: Historical delivery data could be used to train demand forecasting models that predict which packages are most likely to be received on a given day, enabling proactive vehicle load planning based on probabilistic demand rather than a static manifest.

Bibliography

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[3] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.

[4] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 3, pp. 215–225, 1975.

[5] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.

[6] R. Sedgewick, "Implementing quicksort programs," *Communications of the ACM*, vol. 21, no. 10, pp. 847–857, 1978.

[7] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester, UK: Wiley, 1990.

[8] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.

[9] B. L. Golden, S. Raghavan, and E. A. Wasil (Eds.), *The Vehicle Routing Problem: Latest Advances and New Challenges*. New York, NY: Springer, 2008.

[10] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA: Addison-Wesley, 1997.

[11] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.

[12] J. Kleinberg and E. Tardos, *Algorithm Design*. Boston, MA: Addison-Wesley, 2006.

[13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[14] R. C. Prim, "Shortest connection networks and some generalizations," *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.

[15] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.